

Tutorial 3

Night Sky

You observed the night sky on N consecutive nights. On night i , you found that there were exactly $A[i]$ stars visible.

Due to peculiarities of the Solar System, it is known that every star is visible during a continuous segment of nights $[L, R]$ that is every star has 2 integers L and R such that $1 \leq L \leq R \leq N$, and the star is visible on nights $L, L + 1, L + 2, \dots, R$ only.

The input consists of N , the number of nights over which stars were observed, and A , a sequence of N values where $A[i]$ denotes the number of stars observed on night i .

There could be many possible combinations of stars that lead to the same number of stars being observed each night. We are concerned with the total number of unique stars that were seen across those N nights.

Night Sky (cont.)

Let m be the minimum and M be the maximum number of unique stars that you could have observed. Your task is to compute m , M .

(a) $N = 3, A = [2, 1, 2]$

(b) $N = 6, A = [1, 2, 3, 4, 2, 1]$

(c) $N = 12, A = [4, 1, 3, 5, 2, 2, 3, 1, 5, 6, 2, 1]$

Counting Good Arrays

We call an array A of M integers good if $A[i] - A[i - 1] \leq K$ for all $1 < i \leq M$.

Given an array of integers A , C is said to be a subsequence of A if it is possible to delete some (possibly none) elements from A to form C , without changing the order of the remaining elements. For example, the non-empty subsequences of $[1, 1, 3]$ are $[1]$, $[1]$, $[3]$, $[1, 1]$, $[1, 3]$, $[1, 3]$ and $[1, 1, 3]$.

You are given a non-decreasing array A of N integers and the parameter K . Your task is to find the number of non-empty subsequences of A that are good. Notice that the same subsequence may arise multiple times in the array A . Each subsequence should be counted as many times as it appears in A .

Counting Good Arrays (cont.)

For example, for the input $N = 3, K = 1, A = [1, 1, 3]$, there are 4 good non-empty subsequences, which are $[1]$, $[1]$, $[3]$ and $[1, 1]$.

(a) $N = 8, K = 0, A = [1, 1, 2, 3, 3, 3, 4, 4]$

(b) $N = 8, K = 4, A = [1, 2, 3, 4, 5, 6, 7, 8]$

(c) $N = 15, K = 9, A = [1, 1, 2, 3, 4, 4, 5, 6, 8, 10, 10, 10, 11, 12, 14]$

Passcodes

You need to select a passcode consisting of N numeric digits. Each digit can be between 0 and 9, both inclusive, and your passcode may have leading 0s. Therefore, there are 10^N possible passcodes.

Let $D[i]$ denote the i th digit of your passcode.

To make your passcode difficult to guess, you make sure that:

- ▶ No two adjacent digits are equal. That is, for all $1 \leq i < N$, $D[i] \neq D[i + 1]$.
- ▶ For any three consecutive digits, they are not all increasing. That is, there must be no $1 \leq i \leq N - 2$ such that $D[i] < D[i + 1]$ and $D[i + 1] < D[i + 2]$.
- ▶ For any three consecutive digits, they are not all decreasing. That is, there must be no $1 \leq i \leq N - 2$ such that $D[i] > D[i + 1]$ and $D[i + 1] > D[i + 2]$.

Passcodes (cont.)

For a given length N , what is the number of passcodes you can create of that length?

- (a) $N = 3$
- (b) $N = 6$
- (c) $N = 11$

Hidden Array

There is a hidden array $A[1], \dots, A[N]$. There are 2^N subsets of elements in this array. You are given the sum of elements for each of these 2^N subsets in a list S . You have to compute $A[1] + A[2] + \dots + A[N]$.

Hidden Array (cont.)

- (a) $N = 3, S = [-5, -3, -2, 0, 0, 2, 3, 5]$
- (b) $N = 6, S = [0, 0, 1, 1, 2, 2, 3, 3, 3, 3, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 7, 7, 7, 7, 8, 8, 8, 8, 8, 8, 9, 9, 9, 9, 9, 9, 10, 10, 10, 10, 10, 10, 11, 11, 11, 11, 11, 11, 12, 12, 12, 12, 13, 13, 14, 14, 14, 14, 15, 15, 16, 16, 17, 17]$
- (c) $N = 6, S = [-19, -18, -16, -15, -14, -13, -12, -11, -11, -11, -10, -10, -10, -9, -9, -8, -8, -7, -7, -7, -6, -6, -6, -5, -5, -4, -4, -4, -3, -3, -3, -3, -2, -2, -2, -2, -1, -1, -1, 0, 0, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 5, 5, 5, 6, 6, 6, 7, 8, 9, 10, 11, 13, 14]$

Beautiful Pillars

There are N pillars, numbered 1 to N . Each pillar i has height $H[i]$, and is composed of $H[i]$ stone slabs stacked on top of each other. A sequence of pillars is called beautiful if the height of each pillar is one more than the height of the previous pillar. For example, $[3, 4, 5, 6, 7]$, $[10, 11, 12, 13]$ and $[4, 5]$ are beautiful sequences, while $[6, 5, 4, 3]$, $[2, 4, 7, 9]$ and $[1, 2, 3, 5, 4]$ are not beautiful. You can perform the following operations on a sequence:

- ▶ Add one stone slab to pillar i . That is, increase $H[i]$ by 1.
- ▶ Remove one stone slab from pillar i , provided this does not remove all stone slabs from the pillar. That is, reduce $H[i]$ by 1 if $H[i] > 1$.

Beautiful Pillars (cont.)

For each of the following sequences of pillars, find the minimum number of operations required to make the sequence beautiful.

(a) $N = 3, H = [2, 4, 6]$

(b) $N = 12, H = [1, 2, 3, 1, 1, 5, 1, 3, 5, 3, 11, 11]$

(c) $N = 20, H =$
[12, 21, 13, 9, 19, 17, 15, 18, 22, 19, 17, 19, 15, 20, 24, 17, 35, 25, 25, 29]

Non-Negative Prefix Sums

Given a list $[x_1, x_2, \dots, x_n]$ of integers, a prefix of length j , for $1 \leq j \leq n$, is the list $[x_1, \dots, x_j]$, and the corresponding prefix sum is $x_1 + x_2 + \dots + x_j$.

The integers in the list may be negative, so a prefix sum could be negative.

A list of length n has n prefixes and n corresponding prefix sums. Our goal is to ensure that no prefix sum is negative. To achieve this, we can flip the signs of some of the negative values in the list. For instance, one way to ensure that all prefix sums are non-negative is to flip the sign of each negative number, making all values non-negative.

Our goal is to identify the minimum number of values in the list whose sign needs to be flipped so that no prefix sum is negative. For example, if the list is $[3, -3, -2, -1, 0]$, we can flip the sign of just one value, -2 , to get $[3, -3, 2, -1, 0]$. The prefix sums of this new list are $(3, 0, 2, 1, 1)$, which are all non-negative.

Non-Negative Prefix Sums (cont.)

For each of the following lists, find the minimum number of values whose signs need to be flipped to ensure that all prefix-sums are non-negative.

(a) $[3, -2, 3, -1, -2, -2, -4]$

(b) $[-15, -12, -10, -13, -2, -3, -17, -19, -5, -9]$

(c) $[-12, -2, -16, -19, -9, -3, -7, -11, -17, -3, -15, -10, -10, -15, -8]$

Cutoffs

Principal Nezu is overseeing choosing students for UA High School. Applicants are divided into two tracks: combat and tech. Each potential student takes part in exactly one of tracks and we know the scores of all the students in combat and tech.

No two combat scores are the same. No two tech scores are the same.

The combat and tech qualifying scores are calculated as follows: You select a qualifying score X for combat and a qualifying score Y for tech. All students who score $\geq X$ in combat are admitted, and all students who score $\geq Y$ in tech are admitted.

Nezu wants the total number of students selected to be exactly K , and you also want to minimize $|X - Y|$ so that both exams seem equally difficult.

If there are multiple valid pairs (X, Y) which minimize $|X - Y|$, you want the pair which maximizes $(X + Y)$.

Cutoffs (cont.)

For each instance, choose the cutoffs.

- (a) Combat Scores: $\{29, 60, 5, 31, 23, 22\}$, Tech Scores: $\{18, 1, 22, 9, 2, 8, 35\}$, $K = 6$
- (b) Combat Scores: $\{21, 10, 9, 45, 7, 12, 14, 47, 29, 17\}$, Tech Scores: $\{29, 5, 8, 46, 1, 27, 13, 7, 32, 2, 15, 12\}$, $K = 11$
- (c) Combat Scores: $\{47, 28, 49, 35, 52, 38, 43, 39, 34, 57, 20, 18, 48\}$, Tech Scores: $\{33, 46, 28, 51, 39, 36, 44, 21, 55, 37, 59, 38, 47, 40\}$, $K = 16$

Amoeba

You have developed an artificial amoeba, and you can control exactly how it divides.

Each individual amoeba can be instructed to divide into A , B , or C amoebas. That is, if you instruct an amoeba to divide into A , this amoeba will disappear, and A different new amoeba will appear.

You start out with K amoeba initially, and you want to give them instructions such that at the end, you have exactly N amoeba left.

Giving an instruction is a costly affair because it requires you to produce some biochemicals, and so you want to give as few instructions as possible. Find and write the minimum number of instructions that you should give to end up with exactly N amoebas. If it cannot be done, write -1 instead.

Note that each instruction is given to a single amoeba, and not all of them together.

Amoeba (cont.)

For example, suppose $K = 1, A = 1, B = 2, C = 3, N = 4$. Then, you can take the single amoeba, instruct it to divide into B (2) amoebas. Now, there are 2 amoebas. Then take one of these amoebas and instruct it to divide into C (3) amoebas. So now, you have 4 total amoebas, which is what we want, and we used 2 instructions. You can check that you can't get 4 amoebas with fewer than 2 instructions, and hence 2 is the minimum, and so the answer is 2.

Find the minimum number of instructions needed for these instances:

(a) $K = 23, A = 7, B = 12, C = 16, N = 114$

(b) $K = 9, A = 7, B = 15, C = 16, N = 76$

(c) $K = 10, A = 9, B = 12, C = 26, N = 138$

Heavy Lists

You are given a list of 0's and 1's: $B[1], B[2], \dots, B[N]$.

A sublist of this list is any contiguous segment of elements—i.e., $A[i], A[i + 1], \dots, A[j]$, for some i and j .

A sublist is said to be Heavy, if the number of 1's in it is at least as much as the number of 0's in it.

We want to partition the entire list into Heavy sublists. That is, a valid partition is a collection of Heavy sublists, such that each of the N elements is part of exactly one of the sublists. We want to find the number of ways of doing so.

Heavy Lists (cont.)

For example, suppose N was 3 and $B = [1, 0, 1]$. Then all the sublists in this are Heavy, except for the sublist which contains only the second element ($[0]$). The various valid partitions are as follows:

- ▶ $([1, 0, 1])$
- ▶ $([1, 0], [1])$
- ▶ $([1], [0, 1])$

Since there are 3 ways to do this, the answer for this would be 3. Compute the number of ways of partitioning the given list into Heavy sublists for the following instances.

- (a) $N = 8, B = [0, 1, 1, 0, 0, 1, 1, 1]$
- (b) $N = 9, B = [1, 1, 0, 0, 1, 0, 0, 1, 1]$
- (c) $N = 9, B = [1, 0, 1, 0, 1, 1, 0, 1, 1]$

Hide and Seek

A group of N children, who are numbered $1, 2, \dots, N$, want to play hide and seek.

In a single round of hide and seek, there will one seeker, and $N - 1$ hiders. Children like to hide and not seek and each child has her own idea of how many times she would like to hide. You will be given for each child i , a number $H[i]$ denoting the number of rounds she would like to be a hider.

She will be satisfied only if she gets to be a hider in at least $H[i]$ rounds.

Hide and Seek (cont.)

For example, suppose $N = 4$, and $H = [1, 3, 2, 1]$. Here is one way to satisfy them all. In Round 1, Child 1 is the seeker, and in Rounds 2 and 3, Child 4 is the seeker. Then Child 1 has been a hider in 2 Rounds, Child 2 has been a hider in 3 Rounds, Child 3 has been a hider in 3 Rounds, and Child 4 has been a hider in 1 Round. Thus, they are all satisfied. You can check it is not possible to satisfy all of them in fewer than 3 Rounds.

Your aim is to determine the least number of rounds that needs to be played so that every child is satisfied. For the example in the previous paragraph, the answer is 3.

(a) $N = 7, H = [6, 13, 9, 5, 15, 8, 9]$

(b) $N = 12, H = [6, 7, 7, 8, 9, 9, 9, 9, 9, 9, 9, 9]$

(c) $N = 15, H = [131, 135, 130, 138, 132, 140, 137, 133, 131, 137, 138, 132, 135, 136, 134]$

Questions on Binary Search

Peak

Given an array A , an element $A[i]$ is a peak if it is not smaller than its neighbor(s): $A[i] \geq A[i - 1]$ and $A[i] \geq A[i + 1]$ with imagining $A[-1] = A[n] = -\infty$.

We want to find a peak.

Peaks 2D

Given a matrix M of numbers, we want to find a peak that is: an entry that is not smaller than its (up to) 4 neighbors.

Machine

A factory has n machines which can be used to make products. Your goal is to make a total of g products. For each machine, you know the number of seconds (k_i for machine i) it needs to make a single product. The machines can work simultaneously, and you can freely decide their schedule. What is the shortest time needed to make g products?

Square Roots

Without calling the inbuilt `sqrt` function, can you implement a function `square_root(n, e)` that returns a such that $a^2 - e \leq n^2 \leq a^2 + e$.

When Is a Problem Solvable by Binary Search?

A problem is killable via Binary Search if:

- ▶ There exists a way to (cheaply) query such that half of the search space could be eliminated
- ▶ We have an easy way to exclude the eliminated elements from the search space
- ▶ Sometimes if the reverse of the problem is easier to solve

When Is a Problem Solvable by Binary Search? (cont.)

Note: Often the second property is inherited from an order that can be defined on the search space. In the above, the order is $\text{false} < \dots < \text{false} < \text{true} < \text{true} < \dots < \text{true}$ with respect to “is the time enough to produce g items”.

When Is a Problem Solvable by Binary Search? (cont.)

Further notice that the time complexity of the algorithm is $\mathcal{O}(\log_2(S)(q(n) + e(n)))$ where S is the size of search space, $q(n)$ is the time per query and $e(n)$ is the time to exclude $n/2$ elements from the search space.

Please mind the use of $\mathcal{O}$ and not Θ . For Θ , we would perhaps need to consider

$\mathcal{O}(q(n) + q(n/2) + \cdots + q(1) + e(n) + e(n/2) + \cdots + e(1))$ or solution to the recursion $T(n) = T(n/2) + e(n) + q(n)$.

Carnival (CEIO 2014)

Each of Peter's N friends (numbered from 1 to N) bought exactly one carnival costume in order to wear it at this year's carnival parties. There are C different kinds of costumes, numbered from 1 to C . Some of Peter's friends, however, might have bought the same kind of costume. Peter would like to know which of his friends bought the same costume. For this purpose, he organizes some parties, to each of which he invites some of his friends. Peter knows that on the morning after each party he will not be able to recall which costumes he will have seen the night before, but only how many different kinds of costumes he will have seen at the party. Peter wonders if he can nevertheless choose the guests of each party such that he will know in the end, which of his friends had the same kind of costume. Help Peter!

